폐가전 전처리: 양팔 SO-101과 얇은 분해 공구로 리모컨 건전지 분리하기

실제와 같은 고정 구조(+ 단자 립, – 단자 스프링)에서 공구로 밀고 들어 올려 빼는 바이매뉴얼 조작



80/80	80/80	0/20 · 0/20	32.5 N
주 평가 성공률	추가 검증 성공률	대조실험 (뒤집기만 / 밀기만)	기기에 걸린 최대 접촉력
4기기 × seed 0–19	4기기 × seed 20–39 (개발에 안 쓴 seed)	수거함에 들어간 셀 0/40 · 0/40	(판정 기준 40 N)

2024315782 이호섭 · 제출 2026-11-10

시뮬레이터: SO101-Nexus 0.10.0 (MuJoCo 3.17.0) · 정책: LLM(Claude, Claude Code)이 작성한 스킬 함수 + 상태 머신

코드: `code/` (README 참조) · 프롬프트: `code/PROMPTS_phase1.md`, `code/PROMPTS_phase2.md`

요약

소형 폐가전을 파쇄하기 전에 교체형 원통형 건전지(AA/AAA)를 빼내는 해체 작업을 SO101-Nexus 시뮬레이터에서 정의하고 풀었다. 리모컨 4종(TV, 셋톱박스(AAA), 에어컨, 조명(AAA))의 배터리 칸은 실제 제품처럼 + 끝이 고정 립 밑에, − 끝이 코일 스프링에 고정되어 있어, 커버를 열고 뒤집거나 흔들어도, 셀을 스프링 쪽으로 밀기만 해도 빠지지 않는다(대조실험 0/20, 0/20). SO-101 두 대가 사람 작업자의 두 손처럼 일한다. 주 팔(R)은 리모컨을 지그에 붙이고 래치 커버를 눌러 밀어 연 뒤 리모컨을 붙잡고, 공구 팔(L)은 거치대에서 얇은 분해 공구(T자 갈고리 팁)를 꺼내 notch에 넣어 두 셀을 스프링 쪽으로 밀어 립을 지나게 하고 + 끝을 지렛대로 들어 립 위에 얹는다. 이어서 R이 리모컨을 폐건전지 수거함 위에서 뒤집고 흔들어 셀을 떨어뜨리고, 빈 본체를 소형 폐가전 상자에 넣는다. 정책은 LLM이 작성한 스킬 함수와 상태 머신이며, 측정값 기반 폐루프 보정과 재시도를 포함한다. 고정된 최종 코드로 4기기 × seed 0–19 = 80회 평가에서 **80/80(100 %)**, 개발 중 한 번도 쓰지 않은 seed 20–39 80회에서 **80/80(100.0 %)**을 얻었다.

이 결과에는 두 단계의 시행착오가 있다. **Phase 1**(중간 발표, 팔 1대)에서는 그리퍼가 셀 + 끝을 들어 올릴 수 없어 "밀면 + 끝이 바닥 지지대 때문에 저절로 들리는" 단순화를 썼고, TV 리모컨만 동작했다(중간 발표 10/10, 재측정 7/10, 다른 기기 0/6). **Phase 2**(최종)에서는 이 단순화를 없애고 팔을 2대로 늘려 공구를 쓰게 했으며, 20차례가 넘는 평가–분석–수정 반복으로 4기기 모두를 성공시켰다. 5장에 두 단계의 실패 원인과 개선 전후 수치를 환경 설계 문제와 정책 문제로 구분해 정리했다.

1. Task Definition — 시나리오와 선택 이유
2. Environment Design — 에셋·배치·카메라·초기 자세·기구
3. Policy — 방식과 이유, 구조, 스킬, LLM 사용
4. Evaluation — 조건, 성공률, 보조 지표, 대조실험
5. Failure Analysis — Phase 1·2 실패 원인과 전후 비교
6. Discussion — 잘 된 점, 한계, 개선 방향
부록 — 재현 방법, 파일 구성, 에셋 출처

1. Task Definition

1.1 문제와 범위

소형 폐가전은 파쇄·선별 전에 배터리 같은 위험 부품을 분리해야 한다. 배터리는 크게 커버를 열고 빼는 **교체형**과 접착된 **내장형**이 있다. 내장형은 하우징을 뜯어야 하는 별도 문제라 **범위 밖(향후 과제)**으로 두고, 이번에는 **교체형 원통형 셀(AA, AAA)이 든 리모컨**을 대상으로 한다. 셀 종류가 달라도 "− 끝 스프링 + + 끝 걸림 턱" 방식은 같아서 같은 동작 순서로 뺄 수 있다는 점이 대상 선택의 근거다.

동기에 대해서는 과장을 피한다. 리모컨 건전지 자체가 큰 산업 문제라고 주장하지 않으며(출처를 확인하지 못한 "폐기물 화재의 주원인" 같은 주장은 쓰지 않는다), **파쇄 전 위험 부품 분리 공정의 대표 사례**로서 **접촉이 많은 분해 동작**(래치 커버 해제, 스프링·립 걸림 해제, 공구 사용, 양팔 협조, 뒤집어 빼기)을 다룬다는 데 의미를 둔다.

1.2 시나리오

항목	내용
환경	국내 소형 폐가전 해체 작업장의 작업대(강철 작업대 + ESD 매트). 해체 지그(펜스 2면) 위에 폐 리모컨 1개가 **배터리 면을 위로** 놓여 있다(위치·각도는 seed마다 무작위). 지그 왼쪽에 빨간 **폐건전지 수거함**, 앞쪽에 파란 **소형 폐가전 상자**, 지그 건너편에 공구 거치대. SO-101 두 대: 주 팔 R(작업대 원점), 공구 팔 L(지그 건너편, 마주 봄).
수행 과정	① R: 리모컨을 지그 펜스에 밀어 붙임 // L: 거치대에서 분해 공구를 집음 → ② R: 래치 커버를 **누르면서 밀어** 분리(커버가 손가락에 끼면 펜스 너머에 버림), 리모컨 재정렬 → ③ R: 리모컨 옆면을 잡아 지그에 고정 → ④ L: 공구 팁을 pry notch에 넣고 두 셀을 스프링 쪽으로 밀어 + 끝이 립을 지나게 한 뒤, + 끝을 지렛대처럼 들어 립 위에 얹음(실패 시 최대 3회) → ⑤ L: 물러남 → R: 다시 잡아 들어 올림 → ⑥ R: 폐건전지 수거함 위에서 뒤집고 흔들어 셀을 떨어뜨림 // L: 공구를 거치대에 반납 → ⑦ R: 빈 본체를 소형 폐가전 상자에 넣음.
성공 판별	에피소드 종료 시 자동 판정(`battery_sim/success.py`), 네 조건 모두 만족: (1) 모든 셀의 중심이 폐건전지 수거함 영역 안, (2) 본체 중심이 소형 폐가전 상자 영역 안(여유 10 mm), (3) 상자 안에 셀이 없음(잘못 분류 방지), (4) 에피소드 동안 로봇(양팔 + 손에 든 공구)과 기기(본체·커버) 사이의 접촉점별 법선력 최댓값 ≤ 40 N. 공구 반납과 커버의 최종 위치는 판정하지 않는다(참고 지표).

40 N에 대하여. 표준이나 측정에서 온 파손 기준이 아니라, 기기를 부수거나 과도한 힘으로 해결하는 정책을 막기 위한 **임의의 보수적 상한**이다. 측정 대상은 접촉점 하나의 법선력 최댓값이며 셀과의 접촉은 제외한다. 서보 최대 토크로 쥐면 42.6 N이 나와(Phase 1), 힘 제한 파지를 넣었다.

1.3 선택 이유 (참신성·난이도)

- **기본 제공 태스크가 아닌 실제 공정**: 폐가전 전처리의 수작업(배터리 분리)을 그대로 옮겼다. 주제는 여러 후보(물건 포장, 셀프 계산대 등)와 비교한 뒤 정했다(프롬프트 기록 참조).
- **정밀 접촉**: 셀과 립 사이 틈 0.5 mm, 립 길이 4.5 mm, 공구 블레이드 두께 0.8 mm. 몇 mm 오차로 성패가 갈린다.
- **공구 사용**: 로봇이 공구를 집고, 손 안에서의 공구 자세를 측정해 보정하며 사용한다.
- **양팔 협조**: 한 팔이 잡고 한 팔이 공구질을 하며, 두 팔의 작업 공간이 겹쳐 충돌을 피해야 한다.
- **5자유도 팔의 자세 제약**: SO-101은 관절이 5개라 위치 3 + 방향 3을 동시에 맞출 수 없어, 뒤집기·파지·공구 자세 모두 계획이 필요하다.
- **기구 상호작용과 일반화**: 래치·스프링·립이 있는 기구, 4종 기기(AA 2종, AAA 2종), 무작위 배치.

2. Environment Design

2.1 장면 구성과 근거

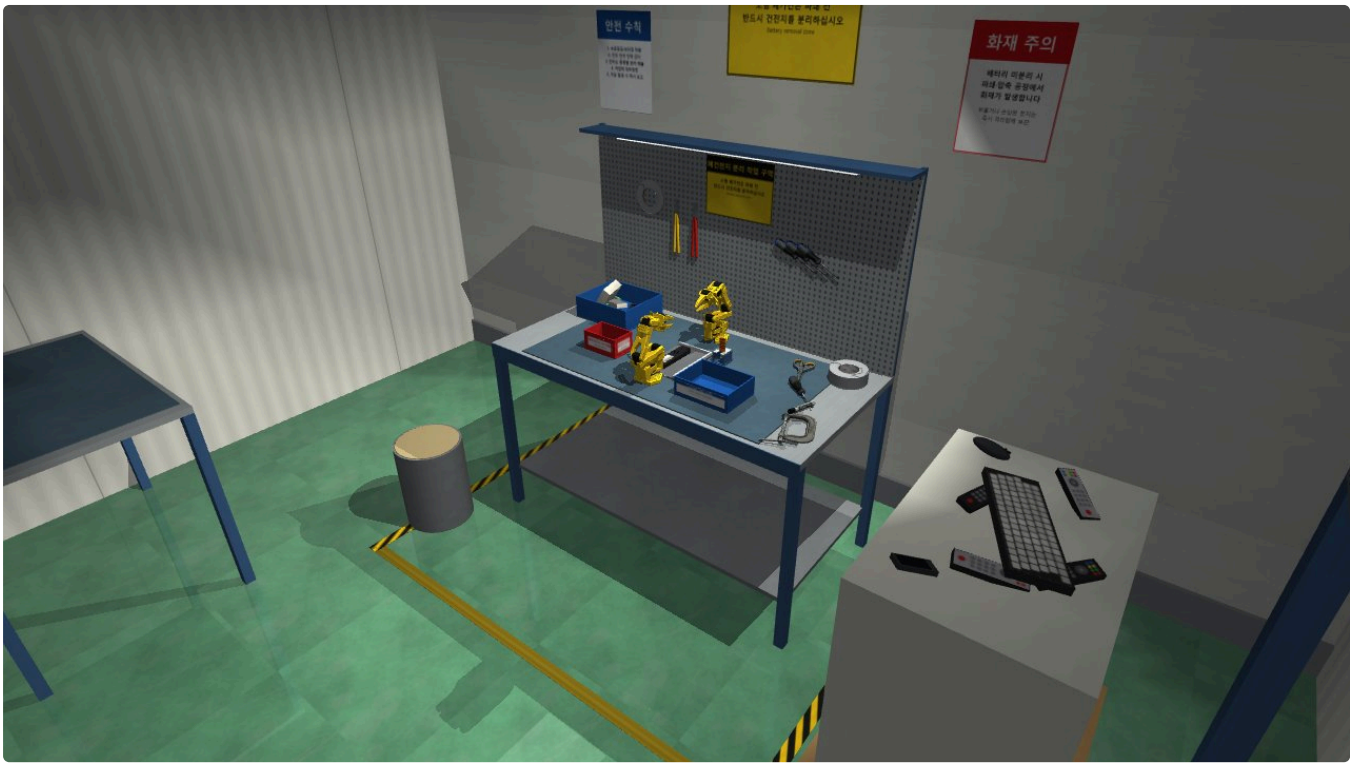


그림 1. 작업장 전경(`wide` 카메라). 녹색 에폭시 바닥, 샌드위치 패널 벽, 안전 포스터, 타공판 공구 걸이, 해체 작업대.

SO101-Nexus 0.10.0이 제공하는 SO-101 MJCF(MuJoCo Menagerie 기반, Apache-2.0)와 STS3215 서보 위치 제어 게인을 그대로 쓰고, 장면은 `battery_sim/scene.py`에서 직접 생성한다. 국내 소형 폐가전 해체 라인을 상정해 꾸몄으며 **실제 현장 사진을 참고한 것은 아니다**.

요소	구성	현실성·타당성 근거
공장 배경	녹색 에폭시 바닥(z = −0.85 m), 샌드위치 패널 벽, 천장 조명, 한글 안전 포스터, 바닥 안전 테이프	재활용·해체 작업장의 전형적 외관
작업대	강철 작업대 + ESD 매트, 타공판(공구 걸이), 선반, 작업등	전자제품 해체 작업대의 표준 구성(정전기 방지)
해체 지그	지그 판 + L자 펜스 2면, 윗면 z = 6 mm	기기를 일정 위치로 맞추는 위치결정 지그. 펜스에 밀어 붙여 위치 오차를 줄인다
폐건전지 수거함	빨간 상자, 11 × 17 × 8.5 cm, 중심 (0.12, 0.30) m, 바닥 두께 12 mm	국내 폐건전지 수거함의 색·라벨. 위치·크기는 5자유도 팔의 뒤집기 도달 범위 분석으로 결정(5.1)
소형 폐가전 상자	파란 상자, 18 × 24 × 7.5 cm, 중심 (0.075, −0.27) m	소형 폐가전 분리배출
공구 거치대 + 분해 공구	포크형 거치대(중심 (0.32, −0.14) m, 판 높이 46 mm). 공구: 고무 손잡이 11 × 22 × 55 mm, 얇은 강철 샤프트, **T자 팁** — 넓은 블레이드 ±8 mm(두 셀의 + 끝면을 동시에 밂), 좁은 목 ±4 mm(두 립 사이 9.6 mm 틈 통과), 두께 0.8 mm, 10° 꺾인 갈고리 발	중간 발표에서 예고한 "얇은 스패너형 분해 공구(spudger)". 공구를 늘 같은 자세로 꺼내고 넣도록 포크형 거치대 사용
장식 공구	드라이버, 가위, 마커(YCB), 테이프 2종, C-클램프(Google Scanned Objects) 3D 스캔 메시, 대기 상자, 톤백	실제 물체 스캔으로 현실감. 모두 팔이 닿지 않는 곳에 정적으로 배치해 태스크에 간섭하지 않음
조명·카메라	천장 확산광 3개 + 작업등. 카메라 9개: `overview, hero, operator, wide, closeup, top, side, bincam, slotcam`	평가·영상·실패 분석용(예: `slotcam`은 pry notch 근접)
로봇	주 팔 R: 베이스 원점, +x 향함. 공구 팔 L: 같은 MJCF를 복제해 이름 앞에 `l_`, 베이스 (0.50, 0, 0), yaw 180°(마주 봄)	사람 작업자의 두 손 역할. 서로 마주 보게 놓아 지그를 사이에 둠
초기 자세	두 팔 모두 HOME = (pan 0°, lift −100°, elbow 40°, wrist_flex 80°, roll 0°), 그리퍼 0	작업 공간 밖으로 접힌 자세

모든 작업 대상(지그, 수거함, 상자, 기기, 공구 거치대)은 SO-101 위쪽 도달 거리(약 0.36 m) 안에 있다. 물리 설정은 timestep 2 ms, implicitfast 적분, elliptic 마찰 원뿔, impratio 10, 제어 주기 20 ms(50 Hz). 장면 전체를 메모리에서 컴파일해(`load_model`) 한글 경로에서도 실행된다(MuJoCo는 비 ASCII 경로 파일을 열지 못함).

2.2 기기 4종

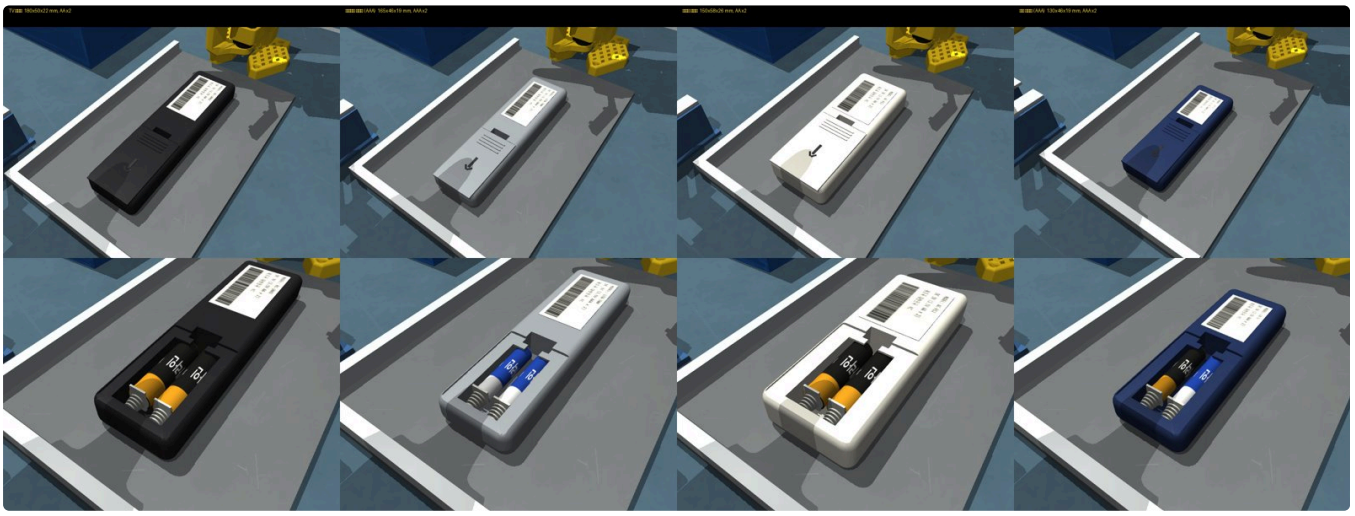


그림 2. 리모컨 4종(위: 커버 닫힘, 아래: 커버 제거). 본체 재질과 스티커 텍스처가 서로 다르다.

기기	길이 × 폭 × 두께 (mm)	셀	셀 간격	− 단자 스프링
`tv_remote` TV 리모컨	180 × 50 × 22	AA ×2 (14.5 × 50.5 mm, 23 g)	16 mm	300 N/m
`stb_remote` 셋톱박스 리모컨	165 × 46 × 19	AAA ×2 (10.5 × 44.5 mm, 11.5 g)	14 mm	150 N/m
`ac_remote` 에어컨 리모컨	150 × 58 × 26	AA ×2	16 mm	300 N/m
`light_remote` 조명 리모컨	130 × 46 × 19	AAA ×2	14 mm	150 N/m

셀 치수·질량은 IEC 규격. 스프링은 모두 0.6 N 예압. 셀 간격과 스프링 강도는 5장의 실패 분석 결과로 정해졌다(AAA 간격 19 → 16 → 14 mm, TV·에어컨 400 → 300 N/m).

2.3 배터리 칸 — 실제와 같은 고정 구조 (핵심 설계)

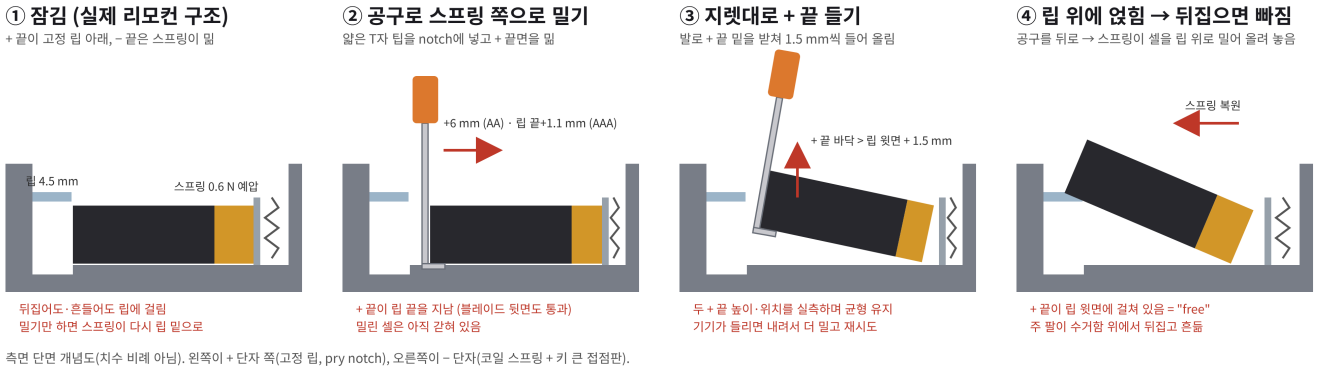


그림 3. 최종 배터리 칸의 고정·해제 원리(개념도). 셀을 빼려면 ②밀기와 ③들기를 모두 해야 한다.

- **커버**: 별도 강체. 옆 레일과 뒷벽 때문에 +x로만 빠진다. 스냅 래치는 weld 구속이며, **커버를 1 N 이상 누른 채 +x로 1.5 N 이상 미는 상태가 10 step(20 ms) 연속**될 때만 환경 코드가 해제한다(실제 래치 탭처럼 누르면서 밀어야 열림).
- **− 단자**: 코일 스프링 플런저(slide joint + stiffness), 예압 0.6 N, 행정 10 mm, 포켓 높이만큼 키 큰 접점판, 금속 마찰 0.12.
- **+ 단자 고정 립**: 각 셀 + 끝 위를 4.5 mm 덮는 강체 탭. 셀 꼭대기와 립 아랫면 사이 틈 0.5 mm. 두 립 사이 간격 **9.6 mm < 가장 가는 셀(AAA 10.5 mm)**이라 뒤집어도 셀이 빠지지 않는다.
- **pry notch**: + 끝 뒤 폭 18 mm, 길이 10 mm, 바닥 3 mm 낮음. 홈(recess)이 셀 밑으로 16 mm 이어져 공구의 넓은 갈고리 발이 두 셀 + 끝 밑까지 들어간다(실제 배터리 홀더의 손톱·드라이버 홈).
- **포켓**: 깊이 = 셀 지름 + 1.9 mm. 셀은 바닥에 평평하게 놓인다(Phase 1의 "저절로 들리게 하는 바닥 지지대" 삭제).
- **접촉**: 11–23 g짜리 가벼운 부품은 `solref 0.005`(딱딱한 접촉). MuJoCo 기본(질량 비례 연성 접촉)으로는 공구가 셀 속으로 수 mm 파고들었다. 셀 마찰 0.25, 포켓 옆벽 0.2.

결과적으로 셀을 빼려면 + 끝을 립 길이보다 더 스프링 쪽으로 밀고(AA 6.2 mm, AAA 립 끝 + 1.1 mm), **그 상태로 + 끝을 립 윗면보다 위로 들어야** 한다. 밀기만 하면 스프링이 셀을 립 밑으로 다시 밀어 넣고, 뒤집기만 하면 립에 걸린다. 이를 두 가지로 검증했다: (a) 공구 없이 기기를 뒤집어 놓고 ±3 g로 흔드는 보유력 시험(`dev_tools/retention_test.py`, 4기기 모두 셀이 칸 안에 남음, 셀 이동 최대 6.1 mm — 판정 기준 30 mm), (b) 실제 정책의 대조실험(4.4절, 0/20 · 0/20).

2.4 초기 상태 무작위화와 안정성 확인

seed마다 리모컨(+ 커버 + 셀을 한 덩어리로)을 공칭 위치 (0.205, 0.005) m 기준으로 **dx ∈ [−20, 0] mm, dy ∈ [−12, +22] mm, yaw ∈ [−0.25, +0.25] rad(±14.3°)** 균일 무작위로 놓고 0.5 s 안정화한 뒤 시작한다. 마찰·질량은 고정. 같은 seed는 같은 초기 상태를 만들고 정책이 결정적이므로 결과가 재현된다. 안정화 후 셀·커버·본체가 의도한 자리에 안정적으로 놓이는지는 디버깅 추적에서 공구 작업 직전 측정한 셀 상태(`cell_state`: + 끝 이동 0.0 ± 0.1 mm, 기울기 0 ± 0.5°)로 확인했고, 공구 없이 뒤집어 흔드는 보유력 시험(2.3)으로 고정 상태도 확인했다.

3. Policy

3.1 방식과 이유: LLM이 작성한 스킬 함수 + 상태 머신

과제가 허용하는 "LLM이 작성한 직접 작성 정책"이다. 스킬 함수(저수준 동작)와 스킬 호출 코드(상태 머신)를 모두 LLM이 생성했고, 학생이 매번 실행해 검증했다. **매 step의 저수준 action을 LLM이 출력하지 않으며 평가 중 LLM 호출은 없다**(결정적·재현 가능). 이 방식을 택한 이유를 두 가지로 나누어 적는다.

- **강화학습·모방학습이 어려운 이유**: 에피소드가 길고(약 70 s, 제어 약 3,500 step) 성공 보상이 끝에만 있다. 래치(누르며 밀기), 립(밀고 들기), notch처럼 좁은 조건을 무작위 탐색으로 찾기 어렵다. 양팔·공구 시연 데이터를 사람이 모으기도 어렵다. GPU 없이 에피소드당 실제 30–40 s가 걸려 학습 예산이 부족하다.
- **스크립트가 맞는 이유**: 단계가 뚜렷해 스킬로 나누기 쉽고, mm 단위 정밀 접촉을 기하로 직접 다룰 수 있으며, 단계 이름이 로그에 남아 실패 지점을 추적하기 쉽다.

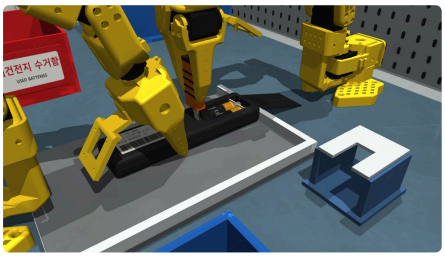
LLM 사용 정보. 도구: Anthropic Claude의 에이전트형 코딩 도구 **Claude Code**. 모델명과 버전은 `code/PROMPTS_phase1.md` 머리말에 기록했다(두 단계 모두 같은 모델). 프롬프트 원문 전체: Phase 1 대화 기록 35개 항목(`PROMPTS_phase1.md`, 도구 알림 포함), Phase 2 사용자 프롬프트 8개(`PROMPTS_phase2.md`). 생성된 코드: `code/` 전체. 대화는 "한 번의 프롬프트"가 아니라 여러 턴이며, Claude가 코드를 작성·실행하고 시뮬레이션 로그·접촉력·렌더링을 보고 원인을 찾아 고치는 방식이었다. 학생은 목표·범위·방향(예: "양팔로", "공구를 써서", "실제와 같은 구조로", "80/80이 될 때까지")을 지시하고, 결과와 설명을 검토했다.

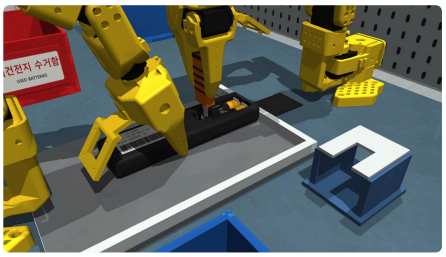
3.2 구조

- **인지**: 시뮬레이터 상태값(기기·셀·공구의 자세, 접촉력, 그리퍼 각도)을 카메라·촉각 센서 대신 쓴다(한계, 6.2).
- **IK**: 감쇠 최소제곱(DLS, `ik.py`). TCP 또는 공구 팁 위치 + 지정한 축 방향(접근 방향, 조 방향)만 맞춘다(5자유도).
- **저수준 제어**: 관절 위치 서보(STS3215 게인). 그리퍼는 서보 힘 제한(잡은 기기 18–30 N)과 점진 파지(`compliant_close`).
- **동시 실행**: 모든 스킬은 파이썬 제너레이터(제어 tick마다 `yield`). `run_par(g1, g2)`가 두 팔의 스킬을 tick 단위로 번갈아 진행해 양팔이 실제로 동시에 움직인다(예: R이 커버를 여는 동안 L이 공구를 집음).
- **상태 머신**: 아래 표의 단계를 순서대로 실행. 각 단계는 측정값으로 성공을 판정해 기록하고, 실패하면 정해진 재시도·복구를 한다.

단계(스킬)	팔	하는 일	피드백·판정
`push_to_fence`	R	닫은 그리퍼로 리모컨 윗면을 눌러 마찰로 끌어 앞 끝을 펜스에 붙임	측정한 펜스 간격·yaw
`pick_tool`	L	거치대에 낮게 −x 방향으로 접근, 손잡이를 폭(22 mm) 방향으로 집고 4 mm 들어 −x로 빼냄. 손안의 공구 자세를 측정해 저장(`calibrate_tool`)	공구 위치, 파지력 > 5 N
`slide_cover`	R	커버 래치를 누르면서 +x로 밀어 분리. 커버가 손가락에 끼면 펜스 너머에 버림(`drop_cover`)	래치 해제 여부, 커버 이동량
`reseat_on_fence`	R	커버를 미는 동안 밀린 리모컨을 다시 펜스에 붙임(참고용)	펜스 간격
`hold_device`	R	리모컨 뒤쪽 단단한 부분을 폭 방향으로 집어 지그에 고정. 공구 팔과 손목이 부딪히지 않게 20° 뒤로 기울여 접근. 5자유도 IK의 위치 오차를 **목표 사전 보정**으로 없앰(고정 조가 옆면 1 mm 바깥에 오도록). 움직이는 조에 힘이 없으면 2 mm 바깥으로 재파지	움직이는 조 접촉력, 파지력
`pry_cells`	L	3.3절. 최대 3회	셀 + 끝 위치·높이·기울기(실측)
`tool_retreat`	L	공구를 기기에서 떼어 거치대 입구 쪽으로(R의 재파지 공간 확보)	—
`grasp_device`	R	놓았다가 위에서 다시 잡아 들어 올림(힘 제한)	들림 높이 > 6 cm
`flip_over_bin`	R	계획된 경로(3.4)로 수거함 위에서 뒤집고 ±0.08 rad로 흔듦	셀이 수거함에 들어갔는지
`return_tool`	L	공구를 포크 거치대에 다시 꽂음(성공 기준 아님)	공구 위치
`drop_in_crate`	R	빈 본체의 긴 축을 수평으로 유지하며 상자 위에 가져가 놓음(격자 탐색)	본체 위치

3.3 공구 스킬 `pry_cells` (가장 어려운 부분)


(a) 블레이드를 notch에 수직으로 넣음


(b) 두 셀을 스프링 쪽으로 밂

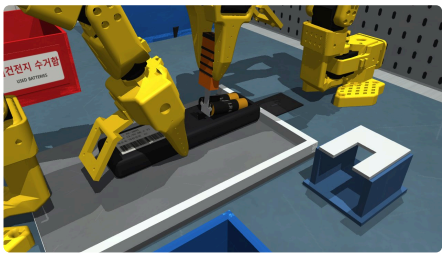

(c) + 끝을 지렛대로 들어 올림

그림 4. TV 리모컨 seed 5의 실제 pry 장면(`closeup` 카메라). 주 팔(왼쪽)이 리모컨을 잡고, 공구 팔(오른쪽)이 공구를 쓴다.

1. **공구 보정**: 손 안의 공구 자세(그리퍼 TCP 대비 팁 위치·방향)를 측정 → 이후 모든 IK는 공구 팁 기준.
2. **접근**: 공구를 10° 뒤로 기울인 채 notch 위로(wrist_flex ±95° 한계 때문). notch 위에서의 수직 자세가 도달 가능하면(IK 오차 < 1.5 mm, 얇은 AAA 기기) 위에서 블레이드를 세운 뒤 수직 하강, 아니면(두꺼운 TV·에어컨) 기울인 채 내려가 바닥에서 세운다.
3. **페루프 팁 보정**(`correct`): 실측 팁 위치로 0.3 mm 이내가 될 때까지 다시 겨냥(팔이 막히면 중단).
4. **밀기**: 0.5 mm씩 +x로 밀며 두 셀 + 끝 위치를 측정. 두 셀 모두 목표(AA 6.2 mm, AAA 립 + 1.1 mm)를 넘거나 실측 팁이 9.5 mm에 닿으면 정지. 한 셀만 앞서면, 늦은 셀도 블레이드 뒷면이 립 끝을 지난 위치(립 + 1.2 mm)일 때만 조기 정지. 블레이드 yaw를 측정해 손목 roll로 보정.
5. **들기(지렛대)**: 1.5 mm씩 올리며 공구를 0° → 10°로 점점 기울임. 두 + 끝 높이를 측정해 2 mm 넘게 차이 나면 높이를 유지하고 발을 늦은 셀 쪽으로 옆 이동. 팁의 옆 오차는 감쇠·상한이 있는 적분 보정. 기기가 0.8 mm 이상 들리면(블레이드가 립에 걸림) 다시 내려서 더 밀고 재시도. 셀이 과하게 밀리면 공구를 살짝 뒤로.
6. **립 위에 얹기**: 두 + 끝 바닥이 립 윗면 + 1.5 mm를 넘으면 발을 립 위로 올린 채 −x로 되돌림 → 스프링이 셀을 밀어 + 끝이 립 위에 얹힘.
7. **빠지기**: 내려서 발을 빼고 옆(−y)으로 빠진다(들린 셀 끝을 치지 않게).
8. **판정·재시도**: 두 + 끝 바닥이 커버 아랫면 − 0.5 mm 이상이고 포켓 안이면 성공. 아니면 재시도(한 셀만 빠졌으면 그 셀 쪽으로 1.1 mm 이동, 진입 위치를 notch 뒤쪽으로).

3.4 뒤집기 계획 `plan_flip`

5자유도 팔로 물체를 뒤집을 수 있는 자세는 제한적이다. 무작위 관절 샘플링(12만 개) → 잡은 순간의 그리퍼–기기 관계로 기기 자세 예측 → 상위 200 후보에 대해 roll 경로(5° 단계, 포켓 위치 고정, 관절 점프 ≤ 30°), 운반 경로, 셀 쏟아짐 방지 조건(운반 중 roll ≤ 30°, pitch −30~+20°), MuJoCo 충돌 검사를 통과하는 경로 중 착지점이 수거함 조준점에 가장 가까운 것을 고른다. 이어서 ±0.08 rad로 흔들고, 뒤집힌 채 1.3 s 들어 셀이 빠지게 한다.

3.5 Phase 1 → Phase 2 정책 변화

	Phase 1 (중간 발표, 11/2–11/6)	Phase 2 (최종, 11/5–11/6)
로봇	SO-101 1대	SO-101 2대(주 팔 R + 공구 팔 L)
걸림 해제	손가락 홈에 그리퍼 끝을 넣어 셀을 5.3 mm 밀기만 함. + 끝은 바닥 지지대(0.8 mm) 때문에 **저절로 3–5° 들림**(환경이 대신한 단순화)	얇은 공구로 밀고 **직접 들어 올림**. 바닥 지지대 삭제, 립 + 스프링 + notch
기기 고정	펜스에 붙이기만 함	주 팔이 옆면을 잡아 고정(사람의 다른 손)
단계	fence → cover → reseat → unlatch → grasp → flip → crate	fence // pick_tool → cover → reseat → hold → pry → retreat → grasp → flip // return_tool → crate
결과	TV만 동작(재측정 7/10), 다른 기기 0/6	4기기 80/80(100 %) + 추가 검증 80/80(100.0 %)

4. Evaluation

4.1 평가 조건

- **seed**: 주 평가 seed 0–19(기기당 20개, 과제 권장 20개 이상) × 4기기 = 80 에피소드. 추가 일반화 검증: **개발 중 한 번도 쓰지 않은** seed 20–39 × 4기기 = 80 에피소드.
- **고정 조건**: 최종 코드를 고정한 채 같은 seed, 같은 성공 기준(1.2절), 같은 무작위화 범위(2.4절). 평가 중 코드·파라미터 변경 없음.
- **명령**: `python run_episode.py --seeds 0-19 --device <기기> -q --out result.json` (seed 20–39도 같은 명령).
- **기록 지표**(에피소드별 JSON): 성공, 셀별 수거함 여부, 본체 상자 여부, 상자 안 셀, 최대 접촉력과 발생 위치·시각, 최종 실패 단계, 시뮬레이션 시간, 물리 step 수(2 ms), pry 시도 횟수, 공구 반납 여부, 실행 시간.

4.2 결과 — 주 평가 (seed 0–19)

기기	성공	수거함 셀	최대 접촉력 평균 / 최대 (N)	시뮬레이션 시간 평균 (최소–최대, s)	물리 step 평균	pry 재시도 에피소드	공구 반납
TV 리모컨 (AA)	**20/20**	40/40	25.9 / 26.2	70.8 (68.8–74.2)	35,418	0	18/20
셋톱박스 리모컨 (AAA)	**20/20**	40/40	25.6 / 25.8	69.8 (66.3–75.3)	34,880	0	20/20
에어컨 리모컨 (AA)	**20/20**	40/40	24.3 / 32.5	70.4 (67.8–72.7)	35,184	0	18/20
조명 리모컨 (AAA)	**20/20**	40/40	25.7 / 25.9	69.3 (66.1–75.6)	34,658	0	20/20
합계	**80/80 (100.0 %)**	160/160	25.4 / 32.5	70.1	35,035	0	76/80

4.3 결과 — 추가 검증 (seed 20–39, 개발에 쓰지 않은 seed)

기기	성공	수거함 셀	최대 접촉력 평균 / 최대 (N)	시뮬레이션 시간 평균 (최소–최대, s)	물리 step 평균	pry 재시도 에피소드	공구 반납
TV 리모컨 (AA)	**20/20**	40/40	26.0 / 26.4	71.0 (68.9–74.2)	35,518	0	13/20
셋톱박스 리모컨 (AAA)	**20/20**	40/40	25.6 / 26.0	72.5 (67.3–77.7)	36,252	0	20/20
에어컨 리모컨 (AA)	**20/20**	40/40	25.0 / 32.4	69.9 (68.1–72.6)	34,970	0	16/20
조명 리모컨 (AAA)	**20/20**	40/40	25.6 / 25.9	71.2 (66.4–77.1)	35,624	0	20/20
합계	**80/80 (100.0 %)**	160/160	25.6 / 32.4	71.2	35,591	0	69/80

두 평가 모두 모든 에피소드가 성공했다. 기기에 걸린 최대 접촉력의 최댓값은 32.5 N(기준 40 N)이다. 시뮬레이션 시간은 에피소드당 약 70 s이며 실제 실행 시간은 클라우드 컨테이너에서 평균 약 51 s(4개 평가 동시 실행)였다. 공구 반납 실패는 성공 기준 밖의 참고 지표로, 공구가 포크 거치대에 끝까지 들어가지 않은 경우다.

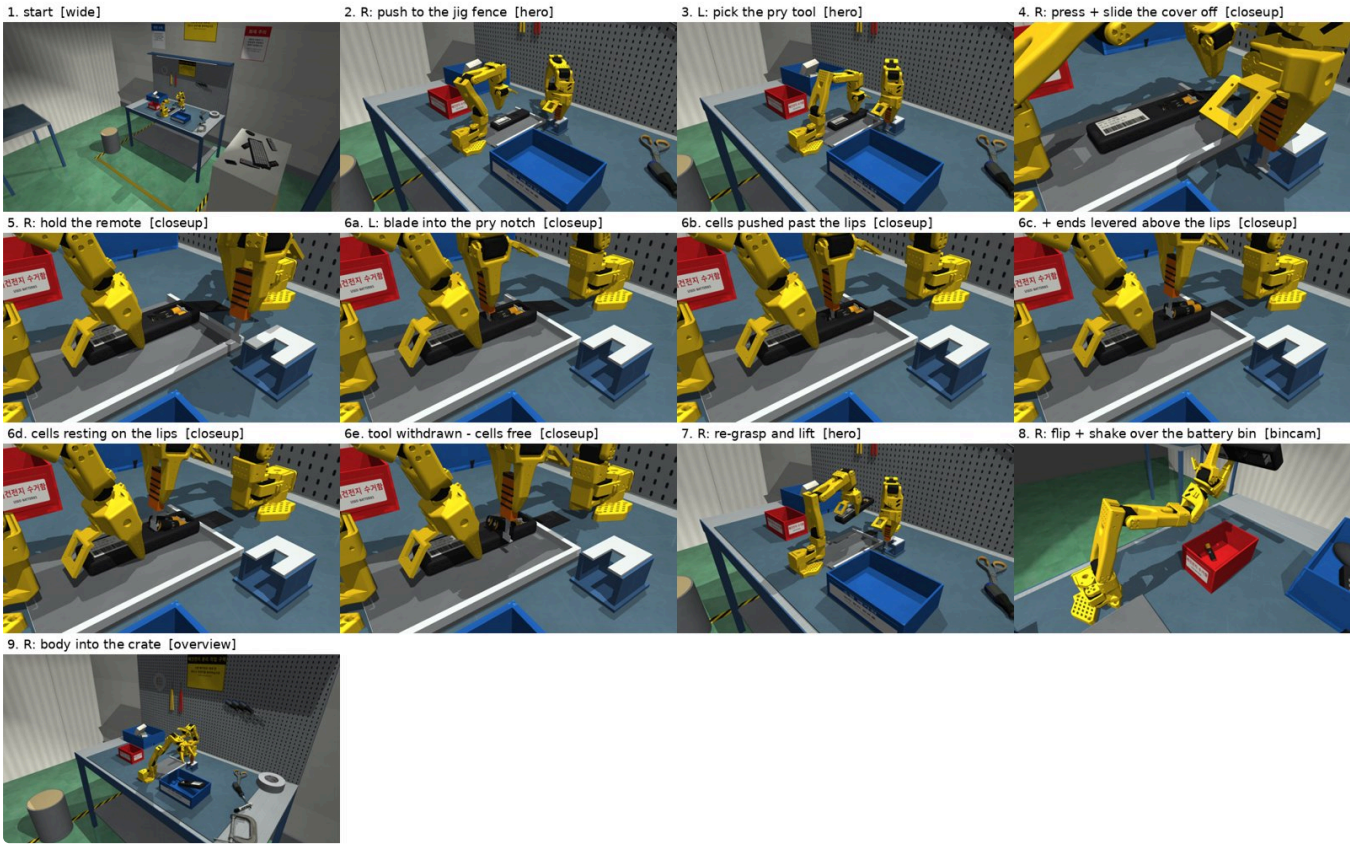


그림 5. 최종 정책의 실제 롤아웃(TV 리모컨 seed 5). 시작 → 펜스 → 공구 집기 → 커버 → 고정 → pry 4단계 → 재파지 → 뒤집기 → 상자. 조명(AAA) seed 15의 같은 그림은 `code/renders/final/`.

4.4 대조실험 — "구조가 실제와 같다"는 근거

같은 seed(0–4 × 4기기)와 같은 성공 기준에서 공구 스킬만 바꿨다(`python ablation.py --mode no_tool|push_only`).

실험	설명	성공	수거함에 들어간 셀
no_tool	공구를 쓰지 않고 커버만 연 뒤 주 팔이 뒤집고 흔듦("그냥 뒤집기")	0/20	0/40
push_only	공구로 셀을 스프링 쪽으로 밀기만 하고 들지 않음(= Phase 1 방식, "밀면 빠질 것")	0/20	0/40
full	전체 정책(밀고 + 들기)	80/80	160/160

Phase 1의 대조실험(걸림 해제 생략)은 성공 0/10이었지만 셀 20개 중 6개가 수거함에 들어가는 "누수"가 있었다(5.1). Phase 2 구조에서는 누수가 0/80이다. 즉 최종 환경에서는 셀이 립과 스프링에 실제처럼 고정되어 "밀고 + 들어 올리기"를 모두 해야만 빠진다.

5. Failure Analysis

과제 요구에 따라 원인을 환경(시뮬레이션 모델·기구 설계 문제)과 정책(스킬·계획·제어 문제)으로 구분한다. 두 가지가 겹친 경우 환경+정책. 실패는 로그(단계별 판정), 접촉 목록(어느 geom이 어느 geom을 몇 N으로 밀었는지), 셀 자세 추적, 단계별 반투명 렌더링으로 분석했다(`code/dev_tools/`).

5.1 Phase 1 (팔 1대, 중간 발표까지)



그림 6. Phase 1 롤아웃(TV seed 5, 팔 1대). 그리퍼로 끌기 → 커버 → 밀어서 해제 → 잡기 → 운반 → 뒤집기 → 상자.

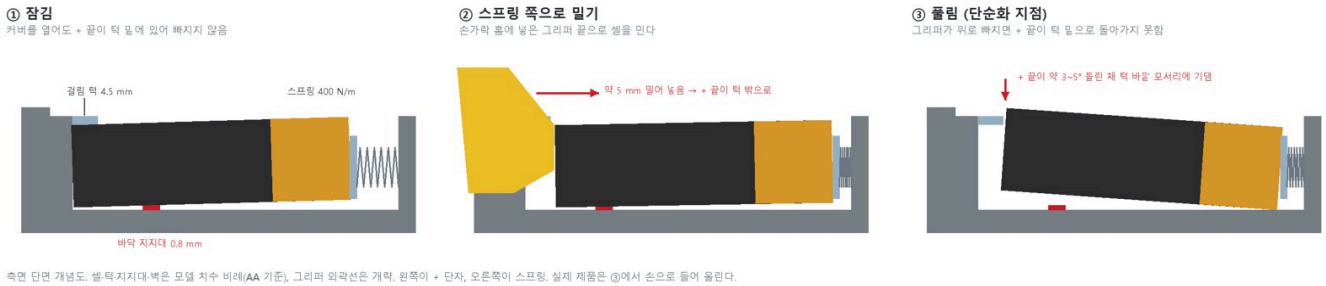


그림 7. Phase 1의 걸림 구조(단면 개념도). ③에서 + 끝이 "저절로" 들리는 것이 단순화 지점 — SO-101 그리퍼 끝은 앞면이 아래로 갈수록 뒤로 물러나는 모양이라 셀 끝면 윗 테두리에만 닿아 들어 올릴 수 없었다.

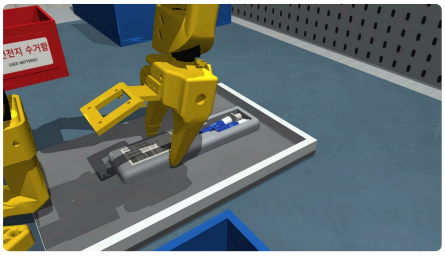
설계 버전별 성공률 (TV 리모컨)

버전	주요 변경	결과	관찰
v1	걸림 턱 없음, 뒤집어 빼기	5/10	턱이 없어 태스크가 너무 쉽고(뒤집기만), 그마저 뒤집기에서 실패
v2	턱 3 mm, 스프링 400 N/m, 해제(밀기) 스킬 추가	9/10	대조실험에서 턱이 약해 해제 없이도 셀이 빠짐
v3	스프링 1000 N/m	0/10	눌러도 해제가 안 되고, 풀린 셀도 떨어지지 않음
v2b	400 N/m, 턱 마찰 0.05, 칸막이 전체 길이	4/5	
v4	턱 4.5 mm, 행정 6.8 mm	10/10	중간 발표 수치
재측정	다기기 리팩터링 후(11/6)	**7/10**	실패 seed 1, 6, 9 모두 뒤집기 단계. 10/10을 낸 코드는 남아 있지 않음

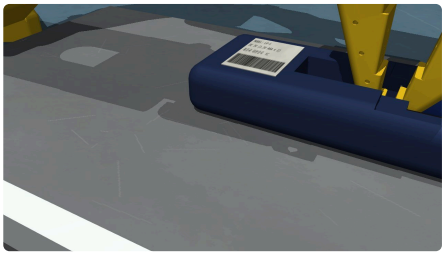
중간 발표 수치의 정정. 중간 발표의 "TV 10/10, 해제 생략 0/5"는 다기기 리팩터링 이전 코드의 결과이며 재현되지 않았다. 같은 TV seed 0–9를 재측정하면 7/10, 해제 생략은 0/10이지만 셀 20개 중 6개가 수거함에 들어갔다(cell0 1, cell1 5). 또 잡는 위치를 1 mm 안팎 바꾸면 성공률이 크게 변했다: grasp_x −29.0 mm 8/10, −30.0 mm 9/10, −30.7 mm(당시 코드) 7/10, −31.5 mm 0/10(셀은 18/20 들어갔지만 본체 투입 실패). 즉 Phase 1의 성공률은 상수에 강하게 의존하는 아슬아슬한 값이었다. 이것이 Phase 2에서 측정값 피드백과 재시도를 넣은 동기다.

Phase 1 실패 원인과 조치

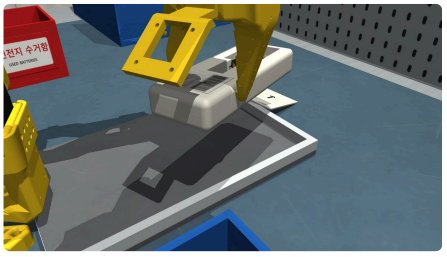
	증상	원인	조치
환경	떨어진 셀이 수거함 바닥 아래로 사라짐	낙하 속도가 2 ms당 4 mm, 바닥 두께 4 mm → 터널링	바닥 12 mm
환경+정책	뒤집을 자세가 없음	수거함 위치 (0.06, 0.24)가 5관절 팔의 뒤집기 도달 범위 밖	도달 가능 자세 분포를 분석해 (0.12, 0.33)으로 이동
환경	턱이 있어도 셀이 빠짐	셀이 옆으로 4 mm 밀려 턱 밖으로 나감	옆 틈 0.25 mm, 칸막이
환경	커버를 밀 때 리모컨이 들림	바닥 충돌 박스를 여러 조각으로 나눈 이음매	AA는 한 장 유지
환경	그리퍼가 홈에 안 들어감	충돌 메시가 간이 박스보다 큼	홈 15.6 × 34 mm로 확대
정책	끌기 실패	베이스 근처에서 팔이 막힘	뒤에서 밀기 대신 윗면 마찰로 끌기
정책	파지력 42.6 N(기준 초과)	서보 최대 토크로 조임	힘 제한 파지
정책	뒤집는 중 리모컨이 빠짐, 이동 중 셀이 먼저 빠짐	무게중심에서 먼 파지, 빠른 회전, 경로 중 기울어짐, 팔 충돌	잡는 위치 조정, 회전 감속, 경로 사전 검증, 충돌 검사
정책	+ 끝을 들어 올릴 수 없음	SO-101 그리퍼 끝 모양	환경 단순화(저절로 들림) — Phase 2에서 공구로 해결
정책	셀이 수거함을 벗어남(Phase 1 미해결)	뒤집을 때 건전지 칸이 수거함 중심에서 24 mm 어긋남. 단, 성공 seed도 같은 계획이라 단독 원인은 아님	Phase 2: 착지점 조준, 수거함 쪽 확장, 5° roll



STB seed 1: 얇은 본체에서 파지 미끄러짐



조명 seed 3: AAA 끝면이 작아 5.3 mm까지 못 밂

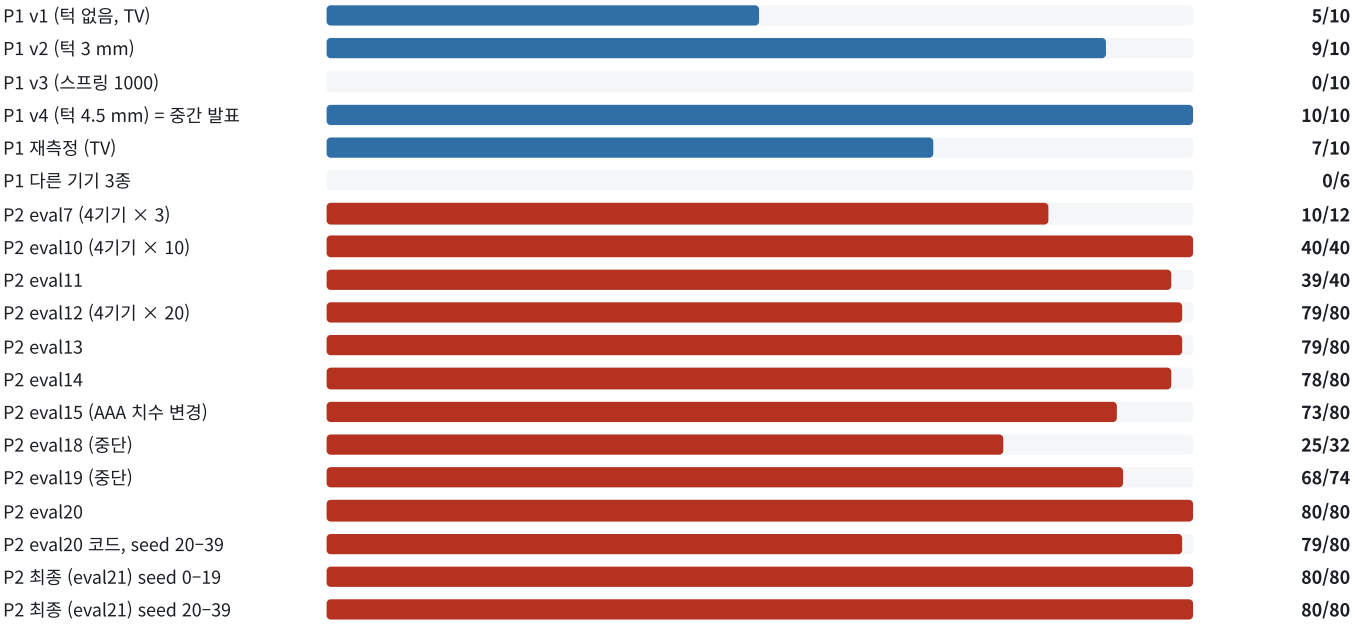


에어컨 seed 2: 잡은 뒤 처짐, 뒤집기 자세 없음

그림 8. Phase 1 다른 기기 실패(0/6). 그리퍼 끝으로 미는 방식이 AAA와 얇은 기기에 맞지 않았다 → Phase 2에서 공구와 양팔로 전환.

5.2 Phase 2 (양팔 + 공구, 최종)

평가 이력 (전후 성공률)



eval16–19는 회귀를 발견하고 중간에 멈춘 평가(분모 = 그때까지 끝난 에피소드). 모든 수치는 `code/dev_tools/_out/evalN/` 로그(저장소에는 최종 결과만 `code/renders/` 에 보관)와 `PROGRESS.md` 작업 로그에 근거.

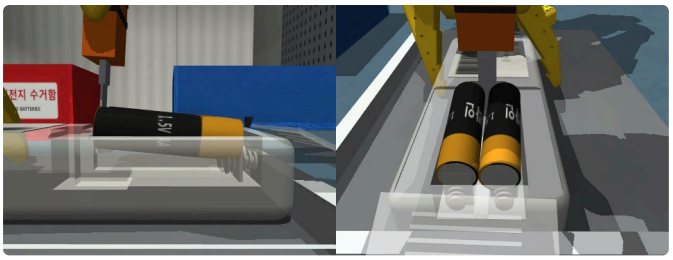
개발 초기: 양팔·공구·실제형 칸을 만들며 해결한 문제

	증상	원인	조치
환경	공구가 셀 속으로 수 mm 파고듦	가벼운 물체에 MuJoCo 기본(연성) 접촉	경량 부품 solref 0.005

환경+정책	거치대에서 공구를 넘어뜨리거나 비뚤게 잡음	높은 접근, 긴 공구	낮은 옆 접근, 포크형 거치대, 짧은 공구, 손잡이 폭 방향 파지
정책	공구 팁 자세 도달 불가	wrist_flex ±95° 한계	10° 기울여 잡기, 들어 올릴 때 점진적 기울임
정책	팁 오차 수 mm	서보 처짐, 손 안 공구 자세	실측 팁 폐루프 보정, 공구 손 안 보정
정책	기울인 블레이드가 셀 윗모서리를 밀어 립에 끼임	밀 때 접촉점이 높음	밀기 전 블레이드 수직
환경	발이 두 셀 사이 V홈을 쐐기처럼 밀어 14 N 옆힘	좁은 notch	넓은 notch + T자 팁
환경	넓은 notch로 셀이 빠져나옴	립 사이가 넓음	좁은 립(틈 9.6 mm < 셀 지름)
정책	커버가 손가락에 끼어 기기 위로 떨어짐	커버를 민 뒤 그냥 놓음	펜스 너머에 버리기(`drop_cover`)
정책	두 팔 손목·카메라 마운트 충돌	작업 공간 겹침	L은 손잡이 폭 방향 파지, R은 20° 기울여 잡기, L이 먼저 물러난 뒤 R 재파지
정책	뒤집기 중 셀이 수거함 밖 / 자세 없음	5자유도 자세 제약	샘플링 기반 계획, 5° roll, 착지점 조준, 수거함 확장(eval7 10/12)
정책	파지력 40 N 초과	서보가 목표까지 계속 조임	서보 힘 제한 18–30 N(eval10 40/40)
정책	운반 중 느슨한 셀이 앞으로 미끄러져 다시 고정됨	운반 자세의 기울기	운반 자세 제약(앞쪽 기울기 ≤ 20°)
환경	TV: 기울어진 셀이 블레이드와 스프링판 사이에서 압축돼 튀어오름	스프링 400 N/m 과다	기기별 스프링(TV 300 N/m)
환경+정책	AAA 셀이 스프링 에너지로 튕겨 나감	가벼운 셀 + 강한 스프링	AAA 150 N/m(예압 0.6 N 유지), AAA는 립 끝 + 1.1 mm만 밀기

최종 단계(80회 평가 반복)에서 분석한 실패 — 대표 사례

	케이스	관찰 → 원인	개선 → 결과
정책	light seed 0 (eval15)	공구가 리모컨을 들어 올리고 리모컨이 12° 회전. 접촉 분석 결과 주 팔의 고정 조가 기기 **옆면이 아닌 윗모서리**에 올라앉아 있었고, 힘 제한기가 누르는 힘을 쥐는 힘으로 오인해 조를 끝까지 열었다 → 기기는 "누르기"로만 고정됨 → 공구를 notch 안에서 세울 때 블레이드 모서리가 notch 벽을 쓸어 기기가 빠짐	움직이는 조 접촉력(`jaw_force`)으로 파지 실패 감지, notch 위에서 먼저 세우기(⑰ 참조) → 해결
정책	light seed 13	들어 올리는 중 공구가 갑자기 3 mm 옆으로 튀고 셀 두 개가 칸 안으로 튕김. 셀 곡면이 발을 옆으로 밀고(15 N) 옆 오차 **적분 보정이 와인드업** → 셀이 풀리는 순간 공구가 튐	적분 감쇠(0.9) + ±2 mm 상한 → 해결
정책	TV/STB seed 0, 6 (eval16, 회귀)	재파지 3회 모두 실패해 그리퍼가 열린 채 고정 실패. 이 자세들에서는 SO-101(5자유도)이 "20° 기울기 + 조 방향"을 **정확히 도달할 수 없어** IK 오차 4–9 mm가 위치 오차로 나타남. 2.5 mm씩 바깥으로 옮기는 재시도로는 해결 불가	원인 확인 후 다른 방법 시도(아래 두 행)
정책	AC seed 0 (eval17, 회귀)	도달 가능한 기울기(0°)로 잡게 했더니 L 팔 추종 오차 20 mm: **R 손목이 L 팔 경로에 들어와 두 팔이 충돌** → 20° 기울기가 필요했던 이유를 재확인	20° 이상만 허용
정책	TV/STB/light seed 0, 6, 9 등	같은 현상(고정 조가 윗모서리에)	**IK 목표 사전 보정**: IK 해의 고정 조 위치가 원하는 위치가 될 때까지 목표를 옮김(남는 오차는 방향으로 1–4°) + 관절공간 이동, 20° 유지 → 20개 시험 중 19개 정상 옆면 파지
환경	AAA 다수 (eval11–18 내내 남던 실패의 **근본 원인**)	들어 올릴 때 두 셀 높이 불균형, 공구가 옆으로 밀림, 셀이 튀거나 걸림. AAA 셀 간격 19 mm에서는 셀 최저선이 |y| = 9.5 mm인데 공구 발은 ±8 mm → 발이 셀의 **안쪽 곡면**을 밀어 올려 큰 옆힘(쐐기, 15 N)과 불균형. AA는 최저선 8 mm = 발 끝이라 거의 수직으로 들어 안정적이었음	AAA 셀 간격 16 mm(AA와 같은 기하) → light 1/3/7/13/16 1회 시도에 균형 있게 성공(eval19 STB 20/20)
환경	light seed 0, 3 (eval19)	거의 다 들었을 때 한 셀이 뒤로 튕겨 기기 밖으로 떨어짐. 접촉점이 y = 7.9–8.0 mm에만 있었음: 셀 중심선(8 mm)이 블레이드 가장자리(±8 mm)와 같아 블레이드가 + 끝면의 **절반만** 막음 → 스프링이 미는 셀이 블레이드 모서리를 돌아 빠짐	AAA 셀 간격 **14 mm**(중심선이 블레이드 안쪽 1 mm). 보유력 시험 통과 → light 0/1/3/5 모두 1회 성공
정책	AC seed 0, 10, 11, TV seed 7 (eval19)	공구 발이 notch가 아니라 립·셀 위에 내려앉음, L 팔 "막힘". light seed 0의 대책(notch 위에서 먼저 세우기)이 두꺼운 기기(AC 26 mm, TV 22 mm)에서는 그 높이의 수직 자세가 wrist_flex 한계로 **도달 불가**(IK 오차 6–16 mm)	수직 자세가 도달 가능할 때만 위에서 세우고 아니면 기울인 채 하강 → 네 seed 모두 1회 성공 → **eval20 80/80**
환경	AC seed 20 (추가 검증 seed, eval20 코드 기준 79/80의 유일한 실패)	들어 올리는 중 셀 1이 갑자기 스프링 쪽으로 19 mm 이동하며 +44°로 뒤집힘(그림 9). 에어컨 셀은 거의 수평으로 통째로 들려 – 끝도 약 6–9 mm 올라가고(성공한 AC 실행에서도 – 끝 중심이 접점판 윗면 근처까지 감), – 끝 접촉이 접점판 위쪽 모서리로 올라가면 블레이드(낮은 + 끝)와 접점판(높은 – 끝)의 압축력 짝힘이 – 끝을 위로 돌림 → – 끝이 접점판을 넘으면 블레이드가 셀을 앞으로 밀어 뒤집힘. 압축력 = 스프링 400 N/m × (예압 + 6.6 mm) ≈ 3 N	에어컨 스프링 **300 N/m**(TV와 같은 값, 짝힘 25% 감소). 보유력 시험 통과 → AC seed 0–39 재평가(4.2–4.3)



(a) t = 30.92 s: 두 셀이 거의 수평으로 들림. 스프링(오른쪽 아래) 위로 – 끝이 올라와 있음　(b) t = 30.98 s: 셀 1의 – 끝이 접점판을 넘어 위로 돌며 튕김

그림 9. AC seed 20 실패의 순간(본체 반투명, 20 ms 간격 재현 렌더링). 원인 분석에 쓴 장면.

교훈. (1) 한 증상만 보고 고친 치수가 다른 실패를 만들었다: AAA + 끝이 홈에 빠지는 문제를 고치려 넓힌 셀 간격(19 mm)이 이후 불균형의 원인이었다. (2) 한 기기에서 효과가 있던 대책(위에서 세우기)이 다른 기기에서는 도달 불가였다. (3) 그래서 고칠 때마다 80회 전체를 다시 돌렸고, 회귀(eval16·17)를 바로 발견해 되돌렸다. (4) 성공률보다 **원인 메커니즘**(접촉점·힘·IK 오차)을 측정해 고치는 것이 효과적이었다.

6. Discussion

6.1 잘 동작한 부분

- 실제 리모컨과 같은 고정 구조(립 + 스프링)에서 **공구를 써서** 셀을 빼는 양팔 조작을 4기기 모두에서 구현했고, 대조실험으로 "밀기만/뒤집기만으로는 안 된다"를 정량 확인했다(0/40 셀).
- 측정값 기반 폐루프(공구 팁 보정, 셀 위치·높이·기울기 피드백, 파지 실패 감지, 힘 제한, 재시도) 덕분에 서보 처짐·공구 미끄러짐·5자유도 IK 오차에 강건했다. 최종 평가 160회 모두 pry가 첫 시도에 성공했다(재시도 0회).
- 제너레이터 기반 동시 실행으로 두 팔이 실제로 병렬 작업(커버 열기 // 공구 집기, 뒤집기 // 공구 반납) — 에피소드 약 70 s.
- 최대 접촉력이 판정 기준 40 N보다 충분히 낮다(4.2).
- 개발에 쓰지 않은 seed에서도 같은 성능(4.3) — 특정 seed에 맞춘 정책이 아님.

6.2 한계

- **인지**: 시뮬레이터 상태값(자세·접촉력)을 그대로 쓴다. 실제 적용에는 손목 카메라 기반 자세 추정과 그리퍼 전류 기반 힘 추정이 필요하다.
- **기기 모델 의존**: 정책이 기기 치수(배터리 칸 위치, 셀 종류)를 안다. 처음 보는 기기에는 치수 정보가 필요하다.
- **민감한 접촉**: 가벼운 AAA 셀과 좁은 공차 때문에 접촉이 혼돈적이다. 같은 설정에서도 시뮬레이터의 미세한 차이(예: 잡는 각도의 1e-4 rad보다 작은 차이)로 결과가 달라지는 사례(AC seed 20)를 확인했다.
- **모델 단순화**: 스프링은 slide joint, 래치는 weld + 힘 조건, 셀은 강체 원통. 셀·기기 치수는 실제 제품 실측이 아니라 대표값이다. 환경 파라미터(셀 간격, 스프링 강도) 일부는 정책이 잘 되도록 조정한 것이며, 모두 실제 리모컨에 있을 법한 범위(AA/AAA 셀 간격, 0.3–0.4 N/mm급 스프링) 안에서 정했다.
- **5자유도 팔**: 위치 + 접근 방향 + 조 방향을 동시에 만족할 수 없는 배치가 있어 파지·공구 자세·뒤집기 계획이 복잡해졌다.
- **공구 반납**은 가끔 포크에 완전히 들어가지 않는다(성공 기준 밖, 4.2 참고 지표).
- 장면은 실제 현장 사진을 참고하지 않았고, 18650 리튬 셀 기기(중간 발표 계획)는 구현하지 않았다.

6.3 개선 방향

- 손목 카메라 기반 기기·셀 자세 추정과 그리퍼 전류 기반 힘 추정으로 상태값 대체.
- 셀을 하나씩 들어 올리는 단일 셀 pry(균형 문제 제거), 들기 중 – 끝 높이 감시.
- 6자유도 팔 또는 회전 지그로 자세 제약 완화.
- 기기 확장: 18650 손전등·휴대용 선풍기, 나사 고정 커버(드라이버 공구 사용).
- 스크립트 정책을 시연 생성기로 써서 비전 입력 정책을 모방학습으로 증류.

부록

A. 재현 방법

```
cd code
uv venv .venv --python 3.12
uv pip install --python .venv/bin/python -r requirements.txt   # Windows: .venv\Scripts\python.exe
python run_episode.py --seeds 0 --device tv_remote                          # 1회, 자세한 로그
python run_episode.py --seeds 0-19 --device light_remote -q --out r.json   # 평가
python ablation.py --mode no_tool --seeds 0-4                               # 대조실험
python live_demo.py --device tv_remote --seed 5                             # 대화형 뷰어 데모
MUJOCO_GL=osmesa python render_rollout_stages.py --device tv_remote --seed 5   # 그림
```

venv는 영문 경로에 둔다. 헤드리스 리눅스 렌더링은 `MUJOCO_GL=osmesa`. 실행 환경: Python 3.12, mujoco 3.17.0, so101-nexus 0.10.0 (Phase 1: Windows 11, Phase 2: Linux 클라우드 컨테이너). GPU 불필요.

B. 파일 구성

경로	내용
`code/battery_sim/scene.py`	장면 MJCF 생성(방, 작업대, 지그, 수거함, 장식, 조명, 카메라, 두 번째 팔, 공구, 거치대)
`code/battery_sim/devices.py`	리모컨 4종(커버, 래치, 스프링, 립, notch, 셀)
`code/battery_sim/env.py`, `ik.py`, `success.py`	양팔 환경·무작위화·래치 로직·접촉력 모니터, DLS IK, 성공 판정
`code/battery_sim/policy.py`	스킬 + 상태 머신(정책 전체)
`code/run_episode.py`, `ablation.py`, `live_demo.py`	평가, 대조실험, 데모
`code/renders/eval_bimanual_*.json`, `renders/heldout/`, `renders/ablation_*.json`	최종 평가 결과(에피소드별)
`code/dev_tools/`	디버깅 도구(스냅샷, 단계별 추적, 접촉 목록, 보유력 시험)
`code/legacy_midterm/`, `records/phase1_single_arm/`	Phase 1 스크립트와 실험 기록(데이터, 그림, 인수인계 문서)
`code/PROMPTS_phase1.md`, `PROMPTS_phase2.md`	LLM 프롬프트 원문 기록
`PROGRESS.md`, `REPORT_MATERIAL.md`	Phase 2 작업 로그, 설계·실패 분석 정리

C. 에셋 출처

- SO-101 로봇 모델: MuJoCo Menagerie(Apache-2.0), SO101-Nexus 0.10.0에 포함된 것을 그대로 사용.
- 장식 메시: Google Scanned Objects(CC BY 4.0) — 테이프 2종, C-클램프. YCB Object and Model Set(ycbbenchmarks.com) — 드라이버, 가위, 마커. YCB는 재배포본(예: Habitat YCB)에 CC BY 4.0으로 표기되어 있으며, 원 배포처의 라이선스 문구는 직접 확인하지 못했다. 모두 SO101-Nexus가 첫 실행 때 내려받는다.
- 나머지(방, 작업대, 지그, 상자, 리모컨, 공구, 텍스처)는 코드로 직접 생성.